SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Serverless Document Processing and Analytics Platform on AWS

An Event-Driven Text Analytics Pipeline using Amazon S3, AWS Lambda, Amazon DynamoDB and Amazon API Gateway

Detailed Project Report

Submitted by

Ram Thokane - 16010123249

Shashwat Jain - 16010123310

B.Tech Computer Engineering, Semester VII

Department of Computer Engineering

K.J. Somaiya School of Engineering, Mumbai

Advanced Cloud Computing

Faculty guide: *[Faculty guide name]*

Academic Year 2026–27 · October 2026

AWS region: ap-southeast-2 (Sydney)

Abstract

Many everyday tasks involve reading a text document and summarising it: how long it is, how it is organised into paragraphs, and which terms occur most often. Doing this by hand is slow and inconsistent, the results are rarely stored anywhere searchable, and running a dedicated server for occasional analysis wastes money and effort.

This project builds a serverless document processing and analytics platform on Amazon Web Services. A browser dashboard written in HTML, CSS and JavaScript asks an Amazon API Gateway HTTP API (DocumentAnalyticsAPI) for a presigned upload URL. The DocumentApi Lambda function validates the request, creates a PENDING record in the Amazon DynamoDB table DocumentAnalyticsV3 and signs a URL for the object key incoming/<documentId>.txt. The browser uploads the file directly to a private Amazon S3 bucket, and the S3 ObjectCreated event invokes the DocumentProcessor Lambda function. DocumentProcessor reads the text and computes the word count, character count, paragraph count and the ten most frequent keywords, then updates the same DynamoDB item. The dashboard reads the results back through the API. Both functions are written in Python with Boto3 and log to Amazon CloudWatch.

The prototype was deployed in ap-southeast-2 (Sydney) and tested on 13–14 October 2026. A direct Lambda test on sample.txt returned 38 words, 314 characters, 4 paragraphs and 10 keywords; files uploaded to the incoming/ prefix were processed automatically by the S3 trigger; GET /documents returned the stored records; and a 32 KB file (hm.txt) passed through the full upload workflow and was processed in the same minute. The report documents the design, implementation, tests, issues found and planned improvements.

Keywords: serverless computing, AWS Lambda, Amazon S3 event notifications, Amazon DynamoDB, Amazon API Gateway, presigned URLs, event-driven architecture, text analytics.

Table of contents

1. Introduction 8
1.1 Background 8
1.2 Problem statement 8
1.3 Aim and objectives 8
1.4 Scope 9
2. Background and related concepts 10
2.1 Serverless computing 10
2.2 AWS Lambda and event-driven execution 10
2.3 Amazon S3 object storage and event notifications 10
2.4 Amazon DynamoDB and NoSQL data modelling 11
2.5 Amazon API Gateway HTTP APIs 11
2.6 Presigned S3 URLs 11
2.7 IAM execution roles and least privilege 11
2.8 CORS and browser-to-cloud communication 12
2.9 Amazon CloudWatch logging and monitoring 12
3. Technology stack 13
3.1 Components 13
3.2 Project at a glance 13
4. System architecture and design 14
4.1 Architecture overview 14
4.2 Synchronous API path 14
4.3 Asynchronous processing path 15
4.4 Data model 15
4.5 API design 16
4.6 Repository structure 16
5. Implementation details 17
5.1 S3 bucket 17

5.2 DocumentProcessor Lambda function 17
5.3 S3 ObjectCreated trigger 18
5.4 DynamoDB table 19
5.5 Text analytics in Python 19
5.6 DocumentApi Lambda function 19
5.7 Lambda execution roles and permissions 20
5.8 API Gateway HTTP API and routes 21
5.9 API Gateway CORS and S3 CORS 21
5.10 Frontend integration through config.js 22
5.11 Dashboard views, upload handling and status polling 22
6. Algorithm and processing logic 24
6.1 Processing steps 24
6.2 Reference implementation 24
6.3 Stable document ID 25
6.4 Status lifecycle 25
7. Testing and observations 27
7.1 Test approach 27
7.2 Observation table 27
7.3 Evidence 28
8. Results and discussion 31
8.1 Demonstrated functionality 31
8.2 Analytics generated 31
8.3 The pending test record 32
8.4 Issues found during testing 32
9. Limitations and future scope 33
9.1 Current limitations 33
9.2 Future scope 33
10. Conclusion 34

10.1 Conclusion 34
10.2 Key learnings 34
References 35
Appendix A: Evidence sources 37

List of figures

Figure 1: System architecture: synchronous API path and asynchronous processing path (editable diagram) 14
Figure 2: S3 bucket serverless-doc-analytics-ram-2026-73921 created in ap-southeast-2 17
Figure 3: DocumentProcessor function overview in the Lambda console (account ID redacted) 18
Figure 4: Deployed DocumentProcessor handler: loops over S3 records and calls process_document 18
Figure 5: S3 event notification: all object-create events, filter incoming/ and .txt, destination DocumentProcessor 19
Figure 6: DynamoDB tables; DocumentAnalyticsV3 (partition key documentId) is used by the final version 19
Figure 7: Execution role observed during development: PowerUserRole with PowerUserAccess (account ID redacted) 21
Figure 8: HTTP API DocumentAnalyticsAPI created (protocol HTTP, regional endpoint) 21
Figure 9: Configuring CORS on DocumentAnalyticsAPI with the origin http://localhost:8080 22
Figure 10: Direct DocumentProcessor test: Executing function: succeeded 28
Figure 11: Console upload of automatic-test.txt (159 B) to the incoming/ prefix 28
Figure 12: CloudShell: aws apigatewayv2 get-api --query CorsConfiguration before the origin was added 28
Figure 13: GET /documents response from the deployed API (01:40 IST), showing stored analytics 29
Figure 14: CloudWatch log streams for /aws/lambda/DocumentProcessor (times in UTC) 29
Figure 15: Dashboard Documents view at 02:03 IST: 5 records, 4 processed and 1 pending 29
Figure 16: Dashboard Overview at 02:08 IST after the hm.txt upload: 6 total, 5 processed, 1 pending, 0 failed 30
Figure 17: Details view for hm.txt: document ID, S3 key, counts and top keywords 30

List of tables

Table 1: Project scope 9
Table 2: Technology components and roles 13
Table 3: Project summary 13
Table 4: DocumentAnalyticsV3 item attributes (as observed) 15
Table 5: HTTP API routes 16
Table 6: Least-privilege permissions per function (backend/iam/) 20
Table 7: Document status values 25
Table 8: Test cases and observed outcomes 27
Table 9: Records observed in DocumentAnalyticsV3 31
Table 10: Implemented features and planned improvements 33
Table 11: Key learnings 34
Table 12: Evidence index 37

1. Introduction

1.1 Background

Cloud platforms now offer managed services that remove the need to provision and maintain servers. In a serverless design the developer writes small functions and connects them to managed storage, databases and APIs; the provider runs the code only when an event arrives and charges for that execution. This model suits workloads that are irregular and event-shaped, such as processing a file when it is uploaded.

Text analysis is one such workload. A student, teacher or analyst who wants quick statistics about a document (its length, its structure and its dominant terms) does not need a permanently running server. The platform described here reacts to each upload, analyses the document once, stores the result and makes it available in a web dashboard.

1.2 Problem statement

Manual analysis of text documents has the following problems:

- Counting words, characters and paragraphs and identifying recurring terms by hand is slow and has to be repeated for every document.
- Manual counts are inconsistent: different people treat punctuation, blank lines and common words differently, so results cannot be reproduced.
- Results are kept in notes or spreadsheets rather than in a persistent, queryable record linked to the original file.
- There is no single place where a user can upload a file, see whether it has been processed and view its analytics.
- A traditional server-based solution would need an always-on machine, operating-system maintenance and capacity planning for a workload that is idle most of the time.

1.3 Aim and objectives

Aim: to design, deploy and test a serverless AWS platform that accepts TXT documents through a web dashboard, analyses them automatically and stores and displays the results.

Objectives:

- **Automated TXT processing:** analyse every .txt file of at most 1 MB placed under the incoming/ prefix of the S3 bucket without manual action.

- **Document analytics:** compute the word count, character count, paragraph count and the top 10 keywords with their frequencies.
- **Persistent storage:** keep one DynamoDB item per document holding its metadata, processing status and analytics.
- **REST-style API:** expose POST /uploads, GET /documents and GET /documents/{documentId} through an API Gateway HTTP API.
- **Dashboard access:** let a user upload a file, list documents, view details and follow processing status from a browser, without AWS credentials in the browser.
- **Secure defaults:** keep the bucket private, upload through short-lived presigned URLs and restrict cross-origin access with CORS.

1.4 Scope

The project covers the storage, processing, database, API and frontend layers of a working prototype. Table 1 summarises the boundary.

Table 1: Project scope

Area	Included
In scope	Private S3 bucket and ObjectCreated trigger; DocumentProcessor and DocumentApi Lambda functions; DocumentAnalyticsV3 table; DocumentAnalyticsAPI HTTP API with CORS; HTML/CSS/JS dashboard; CloudWatch logs
Out of scope	User authentication and authorisation; PDF or other non-TXT formats; files larger than 1 MB; public hosting of the frontend; production monitoring, alarms and load testing

2. Background and related concepts

This chapter explains the cloud concepts the platform depends on and, for each one, how it is applied in this project.

2.1 Serverless computing

Serverless computing is a cloud execution model in which the provider allocates compute only when code runs, scales it automatically and bills per request and execution time. The developer manages functions and configuration but not servers, operating systems or capacity. Serverless designs are usually composed of managed building blocks: functions for logic, object storage for files, a managed database for state and an API front door [13].

In this project: no virtual machines are used. Compute is provided by two Lambda functions, storage by S3, state by DynamoDB and the HTTP interface by API Gateway. When no documents are being uploaded, no compute is running.

2.2 AWS Lambda and event-driven execution

AWS Lambda runs a function in response to an event, such as an HTTP request routed by API Gateway or a notification from S3 [1]. Each invocation receives the event as a JSON document and a context object; for Python the entry point is a handler function, by convention lambda_handler(event, context) [14]. Services such as S3 invoke Lambda asynchronously: the event is queued, and if the function returns an error Lambda retries the invocation [15].

In this project: DocumentApi is invoked synchronously by API Gateway and must answer the browser within the request. DocumentProcessor is invoked asynchronously by S3 for every new object under incoming/ ending in .txt, so processing happens without the browser waiting for it.

2.3 Amazon S3 object storage and event notifications

Amazon S3 stores objects (files) in buckets under keys. A bucket can publish event notifications, for example when an object is created, to destinations including Lambda functions; notifications can be filtered by key prefix and suffix [3]. S3 Block Public Access settings prevent buckets and objects from being made public through ACLs or bucket policies [5].

In this project: the bucket serverless-doc-analytics-ram-2026-73921 holds uploaded documents. One event notification sends all object-create events whose key starts with

incoming/ and ends with .txt to DocumentProcessor. The bucket is not public; browsers can write to it only through presigned URLs.

2.4 Amazon DynamoDB and NoSQL data modelling

Amazon DynamoDB is a fully managed NoSQL key-value and document database [8]. Every item is identified by its primary key; other attributes are schemaless and may contain nested maps and lists. NoSQL modelling starts from the access patterns the application needs, rather than from normalised tables [9].

In this project: the access patterns are simple: fetch one document by ID, and list all documents. DocumentAnalyticsV3 therefore uses documentId (string) as its partition key and stores everything about a document, including the nested analytics map and the keyword list, in a single item.

2.5 Amazon API Gateway HTTP APIs

API Gateway HTTP APIs route HTTP requests to backends such as Lambda functions with low overhead and built-in support for CORS [6], [7]. Each route is a method and path, for example GET /documents/{documentId}, and path parameters are passed to the integration in the event.

In this project: DocumentAnalyticsAPI is a regional HTTP API with three routes, all integrated with DocumentApi. The function inspects routeKey to decide which operation to run.

2.6 Presigned S3 URLs

A presigned URL grants time-limited permission to perform one specific S3 operation on one object. It is generated with the credentials of a principal that already has that permission, and anyone holding the URL can perform only that operation until it expires [4].

In this project: DocumentApi signs a PUT URL for exactly one key, incoming/<documentId>.txt, with the content type text/plain. The browser uploads the file with that URL, so it never holds AWS credentials and the large file body never passes through Lambda or API Gateway.

2.7 IAM execution roles and least privilege

Every Lambda function runs with an execution role, an IAM role whose policies define what the function may do [10]. AWS recommends granting least privilege: only the actions and resources a workload needs [11].

In this project: the minimum permissions were worked out and written as policy documents in backend/iam/ (Section 5.7). During development an execution role with the broader AWS-managed PowerUserAccess policy was observed in the console; replacing it with the scoped policies is listed as an improvement.

2.8 CORS and browser-to-cloud communication

Browsers block JavaScript from reading responses from a different origin unless the server opts in with Cross-Origin Resource Sharing (CORS) headers; for non-simple requests the browser first sends an OPTIONS preflight [17]. Both API Gateway and S3 have their own CORS configuration [7], [18].

In this project: the dashboard is served from http://localhost:8080 and calls two other origins: the API (GET and POST with a JSON content type) and the S3 bucket (PUT through the presigned URL). Both therefore need CORS rules that allow this origin.

2.9 Amazon CloudWatch logging and monitoring

Lambda automatically sends function output and platform log lines to Amazon CloudWatch Logs, into a log group named /aws/lambda/<function-name>, with one log stream per execution environment [12].

In this project: the Python logging module is used in both functions, and the /aws/lambda/DocumentProcessor log group was used to confirm invocations while testing.

3. Technology stack

The platform uses only managed AWS services and a static frontend. Table 2 lists each component and its role.

3.1 Components

Table 2: Technology components and roles

Layer	Technology	Role in the project
Storage	Amazon S3	Private storage of uploaded TXT files; source of ObjectCreated events
Compute	AWS Lambda (Python)	DocumentApi for API requests; DocumentProcessor for text analysis
API	Amazon API Gateway HTTP API	DocumentAnalyticsAPI with three routes and built-in CORS
Database	Amazon DynamoDB	DocumentAnalyticsV3: metadata, status and analytics per document
Logging	Amazon CloudWatch Logs	Lambda execution logs for testing and troubleshooting
Security	AWS IAM	Lambda execution roles and permission policies
SDK	Boto3	S3 and DynamoDB calls and presigned URL generation from Python
Frontend	HTML, CSS, JavaScript	Dashboard with Overview, Upload, Documents, Details and API setup views

3.2 Project at a glance

Table 3: Project summary

Item	Value
Course	Advanced Cloud Computing
AWS region	ap-southeast-2 (Sydney)
S3 bucket	`serverless-doc-analytics-ram-2026-73921`
DynamoDB table	`DocumentAnalyticsV3`
Lambda functions	`DocumentApi`, `DocumentProcessor`
HTTP API	`DocumentAnalyticsAPI`
Supported input	TXT (text/plain), maximum 1 MB (1,048,576 bytes)
Frontend	Static files (index.html, style.css, app.js, config.js) served locally on port 8080

4. System architecture and design

4.1 Architecture overview

Figure 1 shows the architecture. It has two paths. The synchronous path (solid red) is driven by the user: the browser calls the HTTP API, which invokes DocumentApi, which reads and writes DynamoDB and signs upload URLs. The asynchronous path (dashed blue) is driven by S3: a new object under incoming/ triggers DocumentProcessor, which reads the object, analyses it and updates DynamoDB. The browser never talks to DynamoDB; it reaches S3 only with a presigned URL.

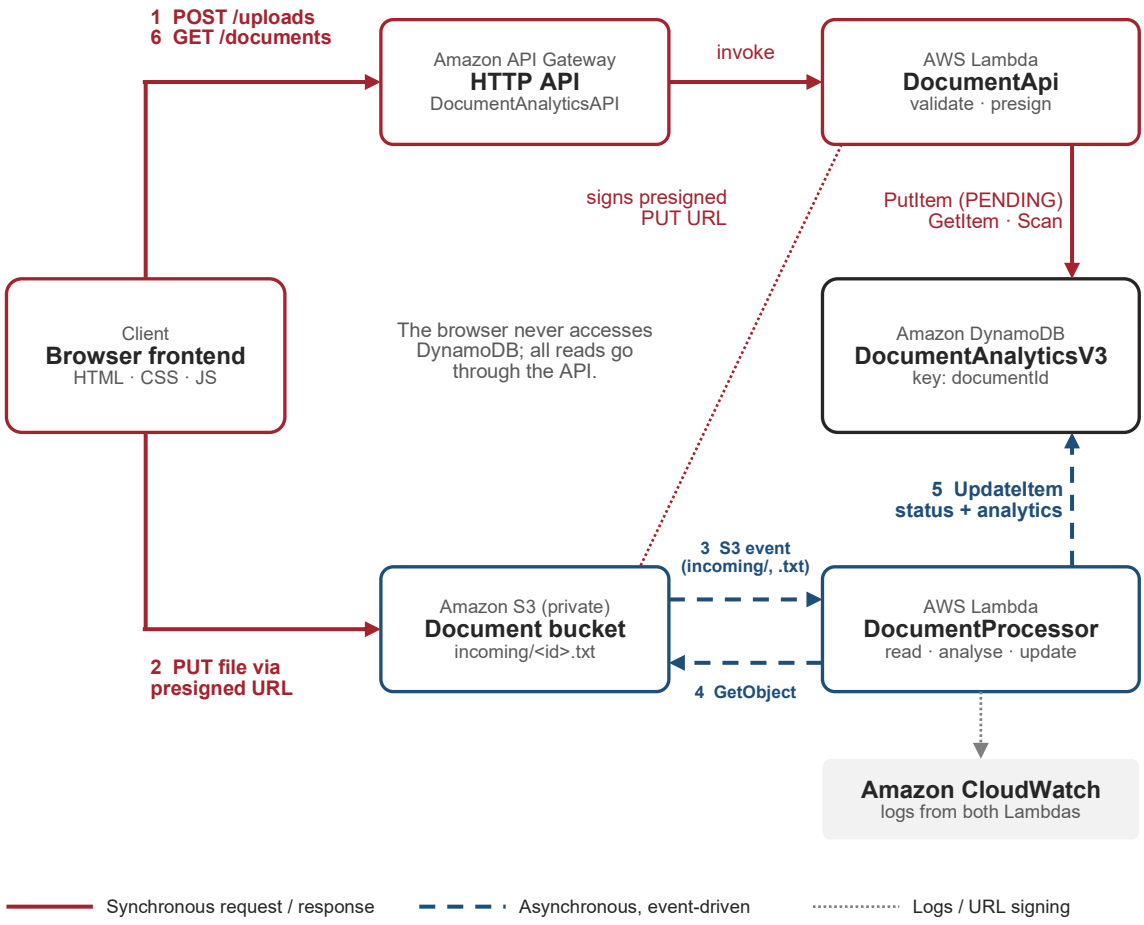


Figure 1: System architecture: synchronous API path and asynchronous processing path (editable diagram)

4.2 Synchronous API path

1. The user selects a .txt file. The dashboard checks the extension, MIME type, emptiness and the 1 MB limit, then sends POST /uploads with fileName, contentType and fileSize.

2. API Gateway invokes DocumentApi. The function validates the request again, generates a new documentId (UUID) and the key incoming/<documentId>.txt, and writes a PENDING item to DocumentAnalyticsV3.
3. DocumentApi returns a presigned PUT URL for that key, together with the documentId.
4. The browser PUTs the file directly to S3 using the URL and the Content-Type header text/plain.
5. The dashboard polls GET /documents/{documentId} until the status becomes COMPLETED or FAILED; the Documents view uses GET /documents.

4.3 Asynchronous processing path

1. S3 stores the object and emits an ObjectCreated event, which matches the incoming/ prefix and .txt suffix filter.
2. S3 invokes DocumentProcessor asynchronously with the bucket name, URL-encoded object key and size.
3. DocumentProcessor decodes the key, derives the documentId from it, downloads the object with GetObject and analyses the text.
4. It updates the same DynamoDB item with status COMPLETED and the analytics, or with FAILED and an error message.
5. Both functions write their logs to CloudWatch.

4.4 Data model

Each document is one item in DocumentAnalyticsV3. Table 4 lists the attributes observed in the GET /documents response of the deployed API (Figure 13).

Table 4: DocumentAnalyticsV3 item attributes (as observed)

Attribute	Type	Written by	Meaning
`documentId`	String (partition key)	DocumentApi / processor	UUID linking the API record, S3 key and item
`filename`	String	DocumentApi	Original file name, for display only
`objectKey`	String	Both	S3 key, e.g. incoming/<documentId>.txt
`fileType`	String	Both	TXT
`fileSize`	Number	DocumentApi	Declared size in bytes
`status`	String	Both	PENDING, COMPLETED or FAILED
`uploadedAt`	String (ISO 8601, UTC)	DocumentApi	Time the upload URL was requested
`processedAt`	String (ISO 8601, UTC)	DocumentProcessor	Time processing finished

Attribute	Type	Written by	Meaning
`expiresAt`	Number (epoch seconds)	DocumentApi	On PENDING items; candidate for DynamoDB TTL
`analytics`	Map	DocumentProcessor	wordCount, characterCount, paragraphCount, topKeywords

topKeywords is a list of maps of the form {word, count}, highest count first. Storing the analytics inside the document item means one GetItem call returns everything the details page needs.

4.5 API design

Table 5: HTTP API routes

Route	Purpose	Response
`POST /uploads`	Validate the file details, create a PENDING item and sign an upload URL	200 with documentId, uploadUrl, key, expiresIn; 400 on invalid input
`GET /documents`	List all document records	200 with items[] and count
`GET /documents/{documentId}`	Return one record including topKeywords	200 with the document; 404 if not found

4.6 Repository structure

The project folder keeps the frontend, the backend reference code and the configuration side by side:

```
serverless-document-analytics/
  index.html               dashboard markup (views and navigation)
  style.css                layout, components, responsive rules
  app.js                   API layer, validation, rendering, routing
  config.js                API base URL, upload rules, polling
  backend/
    document_api/lambda_function.py         DocumentApi reference code
    document_processor/lambda_function.py   DocumentProcessor reference code
    iam/document-api-policy.json            least-privilege policy draft
    iam/document-processor-policy.json      least-privilege policy draft
    s3-cors.json                            bucket CORS rule
    tests/test_backend.py                   offline unit tests (mocked AWS)
```

The functions running in AWS were edited and deployed through the Lambda console. The backend/ folder holds reference versions used to design the logic. The deployed functions store results in a nested analytics map with the attribute names filename and objectKey (Figure 13), whereas the reference code uses flat attributes. The dashboard's normaliseDocument function in app.js accepts both forms.

5. Implementation details

This chapter follows the order in which the platform was built. All resources were created in ap-southeast-2.

5.1 S3 bucket

A general-purpose bucket named serverless-doc-analytics-ram-2026-73921 was created with the default settings, which keep Block Public Access enabled (Figure 2). The bucket was never made public: browsers write to it only through presigned URLs, and DocumentProcessor reads from it through its execution role. Uploaded documents are placed under the incoming/ prefix so that the event filter can target them.

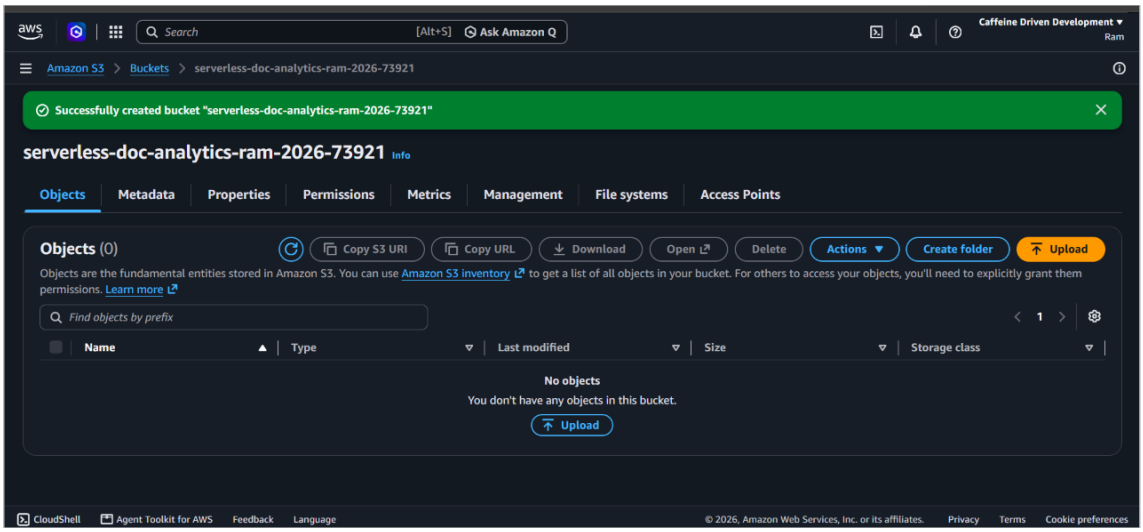


Figure 2: S3 bucket serverless-doc-analytics-ram-2026-73921 created in ap-southeast-2

5.2 DocumentProcessor Lambda function

DocumentProcessor is a Python Lambda function whose handler iterates over the Records in the S3 event, ignores anything whose eventSource is not aws:s3, URL-decodes the object key with unquote_plus and passes the bucket, key and size to a process_document function (Figure 4). Figure 3 shows the function after an update in the console.

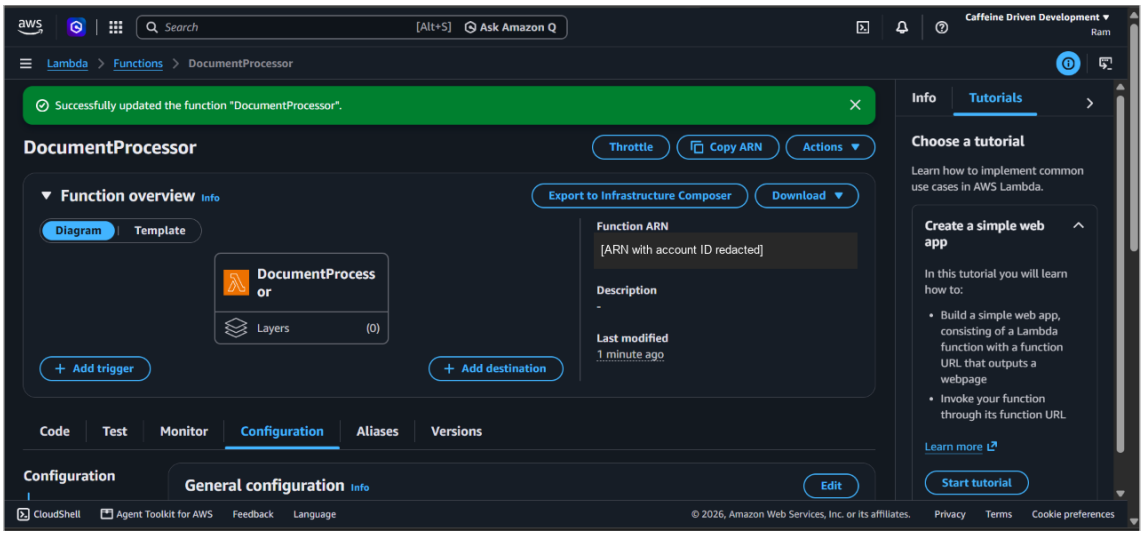


Figure 3: DocumentProcessor function overview in the Lambda console (account ID redacted)



Figure 4: Deployed DocumentProcessor handler: loops over S3 records and calls process_document

5.3 S3 ObjectCreated trigger

An event notification on the bucket sends All object create events with the prefix incoming/ and the suffix .txt to the DocumentProcessor function (Figure 5). The filter has two purposes: it ignores files that are not TXT, and it prevents objects written elsewhere in the bucket from triggering the function [2].

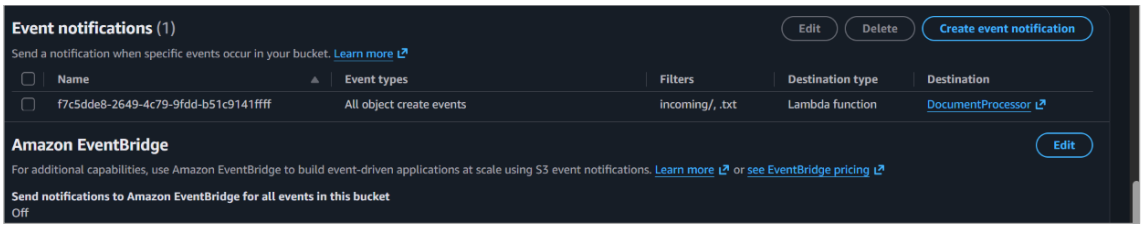


Figure 5: S3 event notification: all object-create events, filter incoming/ and .txt, destination DocumentProcessor

5.4 DynamoDB table

The table DocumentAnalyticsV3 was created with the partition key documentId of type String and no sort key (Figure 6). Two earlier tables, DocumentAnalytics and DocumentAnalyticsV2, remain in the account from previous iterations; the final configuration uses DocumentAnalyticsV3.

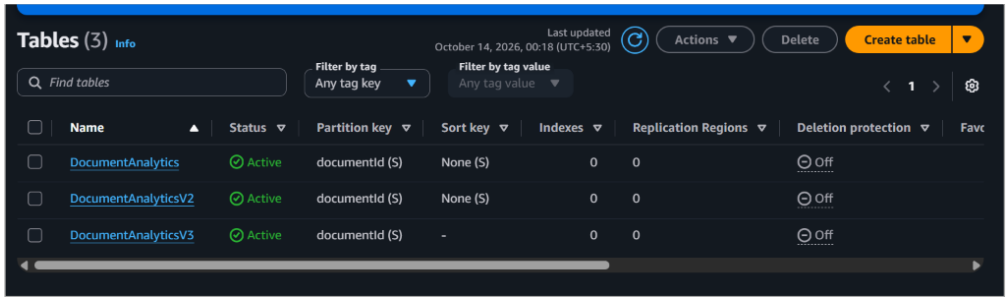


Figure 6: DynamoDB tables; DocumentAnalyticsV3 (partition key documentId) is used by the final version

5.5 Text analytics in Python

The analysis uses only the Python standard library: the re module for tokenising text and splitting paragraphs [16], and frequency counting for keywords. Chapter 6 describes the algorithm step by step.

5.6 DocumentApi Lambda function

DocumentApi handles the three routes. It reads routeKey from the HTTP API event (payload format 2.0) and dispatches to create_upload, list_documents or get_document. Errors from AWS calls are logged and returned as 502, and unexpected errors as 500, so the browser always receives a JSON message. Numbers returned by Boto3 from DynamoDB are Decimal objects and are converted to int or float before json.dumps. The key part of create_upload in the reference code is shown below: the server, not the client, chooses the ID and the key, writes the PENDING item first and only then signs the URL.

```
document_id = str(uuid.uuid4())
key = f"{UPLOAD_PREFIX}{document_id}.txt"      # incoming/<id>.txt
```

```
table.put_item(
    Item={TABLE_KEY: document_id, "fileName": file_name, "s3Key": key,
          "fileSize": file_size, "status": "PENDING",
          "uploadedAt": now, ...},
    ConditionExpression="attribute_not_exists(#pk)", ...)
upload_url = s3.generate_presigned_url("put_object",
    Params={"Bucket": BUCKET_NAME, "Key": key,
            "ContentType": "text/plain", "ContentLength": file_size},
    ExpiresIn=URL_EXPIRES_SECONDS, HttpMethod="PUT")
```

Before this, the function checks that fileName ends in .txt (keeping only the last path segment, so a name like ../a.txt cannot become a path), that contentType is text/plain and that fileSize is an integer between 1 byte and 1 MB. The client file name is stored for display only and never used as an S3 key.

5.7 Lambda execution roles and permissions

Table 6 lists the permissions each function needs, as drafted in backend/iam/. Both functions also need the AWSLambdaBasicExecutionRole managed policy so that they can write CloudWatch logs.

Table 6: Least-privilege permissions per function (backend/iam/)

Function	Action	Resource
DocumentApi	`s3:PutObject`	arn:aws:s3:::serverless-doc-analytics-ram-2026-73921/incoming/*
DocumentApi	`dynamodb:PutItem, GetItem, Scan`	DocumentAnalyticsV3 table ARN
DocumentProcessor	`s3:GetObject`	arn:aws:s3:::serverless-doc-analytics-ram-2026-73921/incoming/*
DocumentProcessor	`dynamodb:UpdateItem`	DocumentAnalyticsV3 table ARN

s3:PutObject is needed by DocumentApi even though it never uploads a file itself: a presigned URL only works if the identity that signed it is allowed to perform the operation. During development, the console showed an execution role named PowerUserRole, trusted by lambda.amazonaws.com, with the AWS-managed PowerUserAccess policy attached (Figure 7). That policy is much broader than Table 6, so attaching the scoped policies is a required step before any production use (Chapter 9).

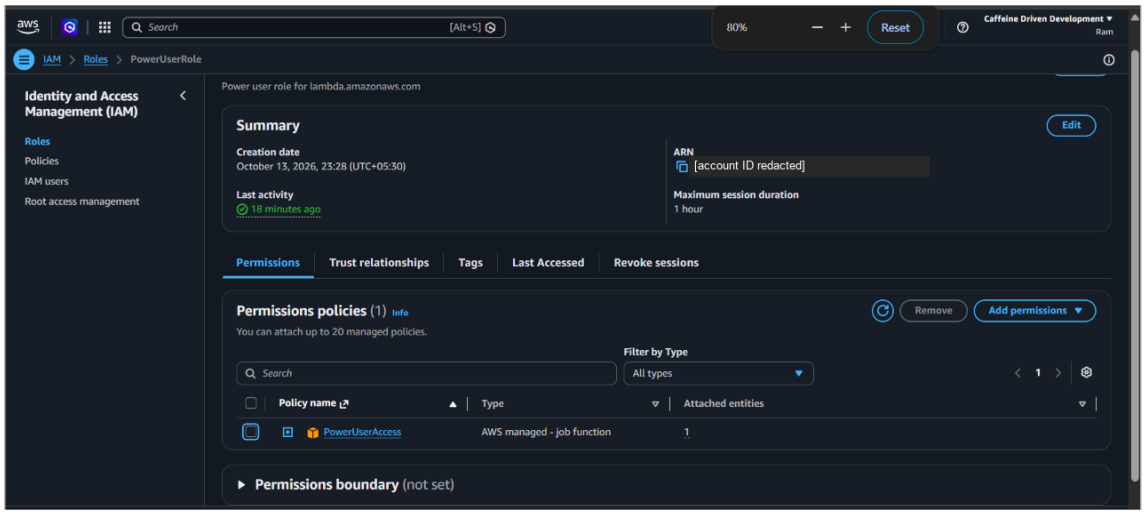


Figure 7: Execution role observed during development: PowerUserRole with PowerUserAccess (account ID redacted)

5.8 API Gateway HTTP API and routes

A regional HTTP API named DocumentAnalyticsAPI was created on 13 October 2026 (Figure 8) with a Lambda integration to DocumentApi and the routes POST /uploads, GET /documents and GET /documents/{documentId}. The routes are reachable at the API's invoke URL (tests 4–9 in Chapter 7), which is placed in config.js.

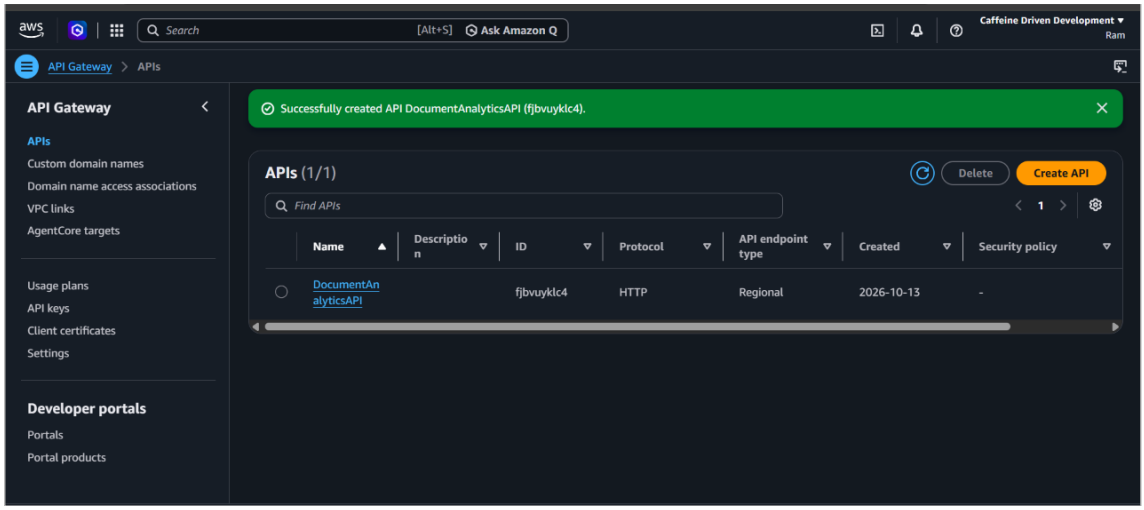


Figure 8: HTTP API DocumentAnalyticsAPI created (protocol HTTP, regional endpoint)

5.9 API Gateway CORS and S3 CORS

CORS was configured in two places. On the HTTP API, http://localhost:8080 was added as an allowed origin, with the methods GET, POST and OPTIONS (Figure 9). Because the API's built-in CORS is enabled, API Gateway answers preflight requests itself and ignores CORS headers returned by the Lambda function. On the S3 bucket,

the rule in backend/s3-cors.json is the configuration a browser upload requires: it allows a PUT with a content-type header from the dashboard's origin:

```
{ "CORSRules": [ {
    "AllowedOrigins": ["http://localhost:8080"],
    "AllowedMethods": ["PUT"],
    "AllowedHeaders": ["content-type"],
    "MaxAgeSeconds": 3000 } ] }
```

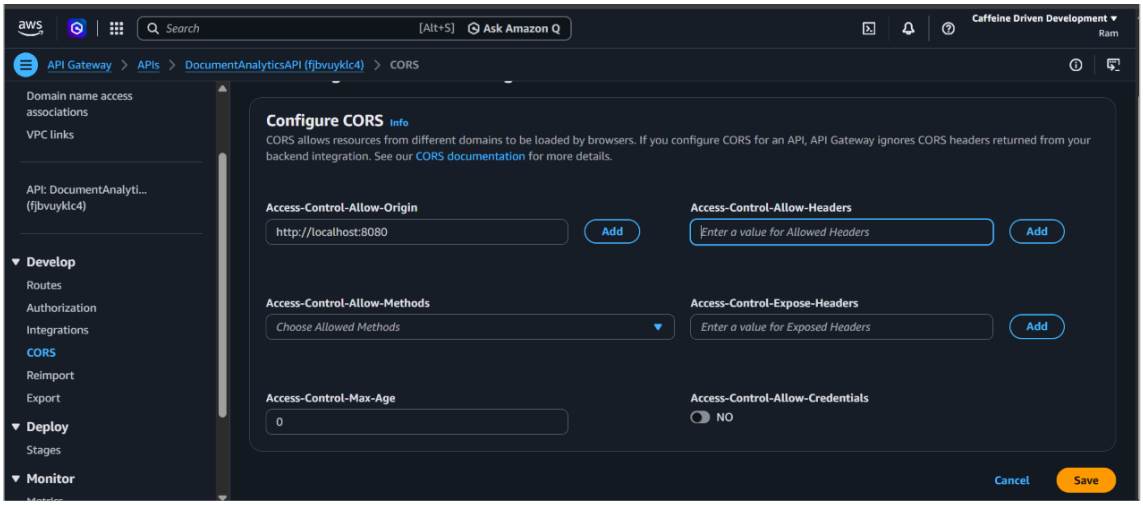


Figure 9: Configuring CORS on DocumentAnalyticsAPI with the origin http://localhost:8080

5.10 Frontend integration through config.js

All environment-specific settings live in config.js, so the rest of the frontend contains no URLs or limits:

```
window.APP_CONFIG = Object.freeze({
  apiBaseUrl: "https://<api-id>.execute-api.ap-southeast-2.amazonaws.com",
  awsRegion: "ap-southeast-2",
  upload:  { allowedExtensions: [".txt"], contentType: "text/plain",
             maxFileSizeBytes: 1024 * 1024 },
  polling: { intervalMs: 3000, maxAttempts: 20 },
  requestTimeoutMs: 15000
});
```

The real API ID is set in the project's config.js and is not repeated here. Until apiBaseUrl is set the dashboard shows an "API not configured" message instead of sample data.

5.11 Dashboard views, upload handling and status polling

The dashboard is a single page with hash-based routing (#/dashboard, #/upload, #/documents, #/documents/<id> and #/setup). All network calls are grouped in an Api object with request timeouts and readable error messages, and all text is inserted with textContent so file names cannot inject HTML.

- **Overview:** counts of total, processed, pending and failed documents and the most recent uploads.
- **Documents:** a table of all records with search by file name, a status filter and sorting.
- **Details:** document ID, S3 key, times, size, the three counts and a bar chart of the top keywords.
- **Upload:** drag-and-drop or file picker, client-side validation, a progress bar during the S3 PUT (XMLHttpRequest is used because it reports upload progress), and status tracking.
- **API setup:** shows whether the last GET /documents call succeeded.

After a successful upload the dashboard polls GET /documents/{documentId} every 3 seconds for up to 20 attempts (one minute). A 404 response is treated as "still processing". Polling stops when the status is COMPLETED or FAILED, or shows a "still processing" message if the limit is reached.

6. Algorithm and processing logic

6.1 Processing steps

For each S3 record, the processing logic in the project's DocumentProcessor code performs the following steps:

1. Decode the object key from the event (S3 keys are URL-encoded, e.g. a space arrives as +) and derive the documentId from it.
2. Check the size reported in the event against the 1 MB limit, then read the object with GetObject, reading at most 1 MB + 1 byte so an oversized object is detected without loading it fully.
3. Decode the bytes as UTF-8 using the utf-8-sig codec, which also removes a UTF-8 byte-order mark (BOM) if one is present.
4. Count words in the text.
5. Count characters as the length of the decoded text, including spaces and line breaks.
6. Count paragraphs by splitting the trimmed text on one or more blank lines (regular expression \n\s*\n) and keeping the non-empty parts.
7. Extract keyword tokens from the lower-cased text with a regular expression, discard tokens of two letters or fewer and tokens in the stop-word set, and count the frequency of each remaining token.
8. Select the 10 most frequent keywords as a list of {word, count} entries.
9. Update the DynamoDB item for this documentId: set status COMPLETED, the analytics and processedAt; or, if processing fails, set status FAILED with an errorMessage.

6.2 Reference implementation

The analysis function in the repository's reference code is short. In this version the word count is the number of whitespace-separated tokens (text.split()), while keywords come from the regular expression [a-z][a-z'-]*, which matches a letter followed by letters, apostrophes or hyphens:

```
STOP_WORDS = frozenset("""a about above after again against all am an and
  any are as at be because been ... with would you your yours yourself
  yourselves also may might must shall""".split())

def analyse_text(text):
    words = text.split()
    paragraphs = [p for p in re.split(r"\n\s*\n", text.strip())
                  if p.strip()]
    tokens = re.findall(r"[a-z][a-z'-]*", text.lower())
    counts = Counter(t for t in tokens
                     if len(t) > 2 and t not in STOP_WORDS)
    return {
```

```
        "wordCount": len(words),
        "characterCount": len(text),
        "paragraphCount": len(paragraphs),
        "topKeywords": [{"word": w, "count": c}   # TOP_KEYWORDS = 10
                        for w, c in counts.most_common(TOP_KEYWORDS)],
    }
```

The method is deterministic and rule-based. It does not use machine learning or natural-language models, so the same file always produces the same result. Ties in keyword frequency keep the order in which the words first appear, which is the behaviour of Counter.most_common [19].

6.3 Stable document ID

The same documentId must connect three things: the PENDING item written by the API, the S3 object key and the item the processor updates. The design achieves this without any extra lookup:

- DocumentApi generates the documentId and builds the key from it: incoming/<documentId>.txt.
- DocumentProcessor reads the documentId back from the key. For keys that do not match this pattern (for example a manual console upload), the reference code derives a deterministic UUID from the bucket and key, so a repeated event for the same object updates one item rather than creating duplicates.
- The processor uses UpdateItem rather than PutItem, and if_not_exists for filename and uploadedAt, so the values written by the API are preserved.

The hm.txt test (Section 7.2) confirms this behaviour in the deployed system: its item has the API-generated UUID as both documentId and key, an upload time written by DocumentApi and a processing time written by DocumentProcessor in the same item.

6.4 Status lifecycle

Table 7: Document status values

Status	Set by	Meaning
PENDING	DocumentApi (POST /uploads)	Upload URL issued; the file has not been processed (it may not have been uploaded)
COMPLETED	DocumentProcessor	Text analysed; analytics stored
FAILED	DocumentProcessor	File rejected or processing raised an error; errorMessage stored

For unexpected errors the reference processor records FAILED and then re-raises the exception, so that Lambda's asynchronous retry still runs; a successful retry sets the

status back to COMPLETED. The FAILED path was not exercised in the observed tests.

7. Testing and observations

7.1 Test approach

Testing was carried out manually on 13–14 October 2026 (times below are IST, UTC+05:30) at four levels: direct Lambda invocation with a console test event, S3-triggered processing using console uploads, API calls, and the browser dashboard running on http://localhost:8080. Evidence comes from console screenshots, the API's JSON output and the dashboard. The repository also contains offline unit tests (backend/tests/test_backend.py) that mock S3 and DynamoDB; their results are not reported here because they were not run as part of the recorded evidence.

7.2 Observation table

Table 8: Test cases and observed outcomes

#	Test case	Expected outcome	Actual observed outcome	Status
1	Direct DocumentProcessor test with event TestSampleDocument (sample.txt)	Function succeeds and returns analytics	Executing function: succeeded (Figure 10); 38 words, 314 characters, 4 paragraphs, 10 keywords stored	**Verified**
2	Console upload of sample.txt (314 B) to incoming/	S3 event triggers processing	Record for incoming/sample.txt COMPLETED with the same counts as test 1	**Verified**
3	Console upload of automatic-test.txt (159 B) to incoming/ (Figure 11)	S3 event triggers processing	Record COMPLETED: 24 words, 159 characters, 1 paragraph	**Verified**
4	GET /documents through the API	JSON list of DynamoDB records	items[] returned with status and analytics of the existing records (Figure 13)	**Verified**
5	POST /uploads for api-test.txt without uploading the file	PENDING item and presigned URL	Item 7c08e696-… with status PENDING and key incoming/7c08e696-….txt; stays PENDING	**As designed**
6	Retrieve API CORS configuration (CloudShell) and correct it	Allowed origin and methods	Methods GET, POST, OPTIONS and MaxAge 300 returned, no origin listed (Figure 12); origin added; dashboard then connects	**Verified**
7	Dashboard Documents view at 02:03 (browser)	Records listed from the API	5 records: 4 Processed, 1 Pending; status pill "API connected" (Figure 15)	**Verified**
8	Upload of hm.txt (32 KB) through POST /uploads and presigned PUT at 02:05	Processed automatically and shown in dashboard	Uploaded and processed at 02:05: 3,388 words, 32,746 characters, 156 paragraphs; Overview shows 6 total, 5 processed, 1 pending, 0 failed (Figures 16, 17)	**Verified***
9	Details view, GET /documents/{documentId}, for hm.txt	Single record with keywords	Details page shows IDs, times, counts and the keyword chart (Figure 17)	**Verified**
10	Reject a non-TXT or > 1 MB file on the deployed API; FAILED path; load	400 / FAILED status	Not tested	**Not tested**

* Test 8 is confirmed by the DynamoDB record (an API-generated UUID key with both an upload and a processing time) and by the dashboard. The CloudWatch log for this invocation was not captured.

7.3 Evidence

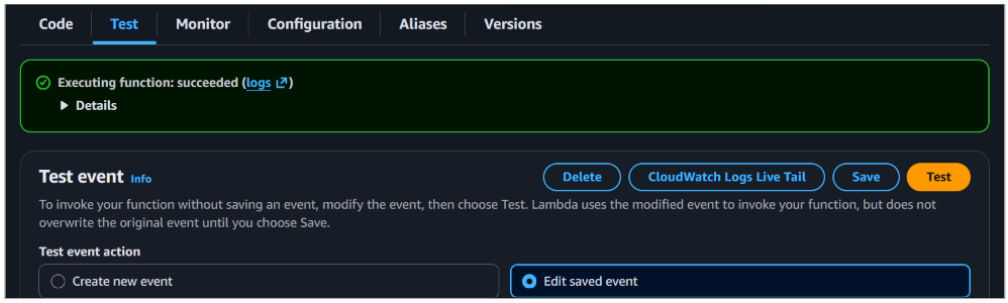


Figure 10: Direct DocumentProcessor test: Executing function: succeeded

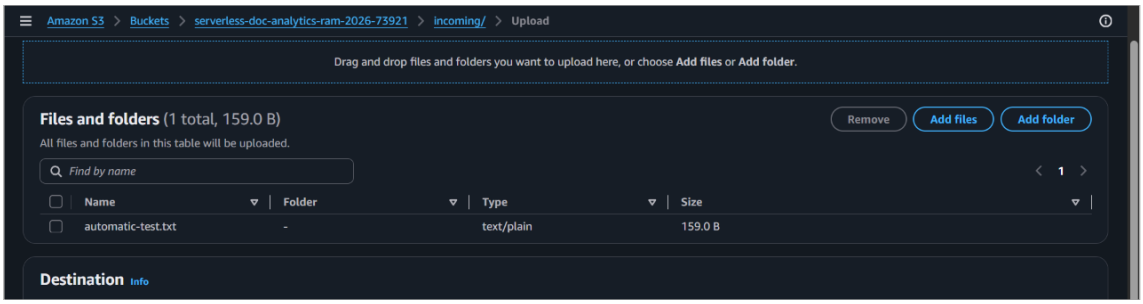


Figure 11: Console upload of automatic-test.txt (159 B) to the incoming/ prefix

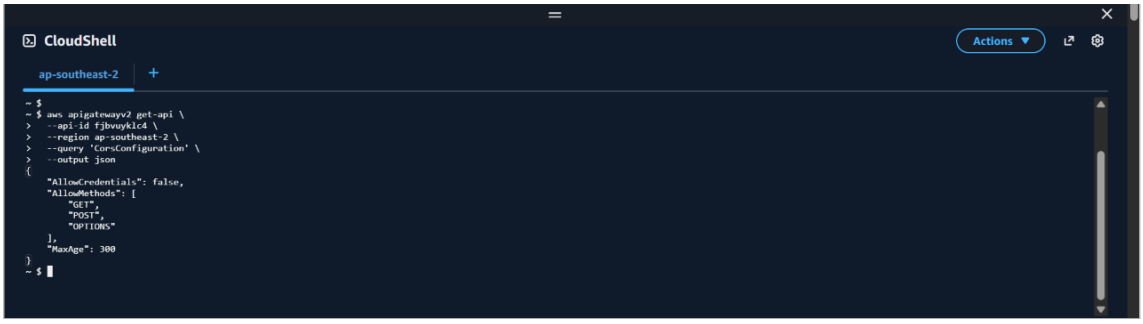


Figure 12: CloudShell: aws apigatewayv2 get-api --query CorsConfiguration before the origin was added

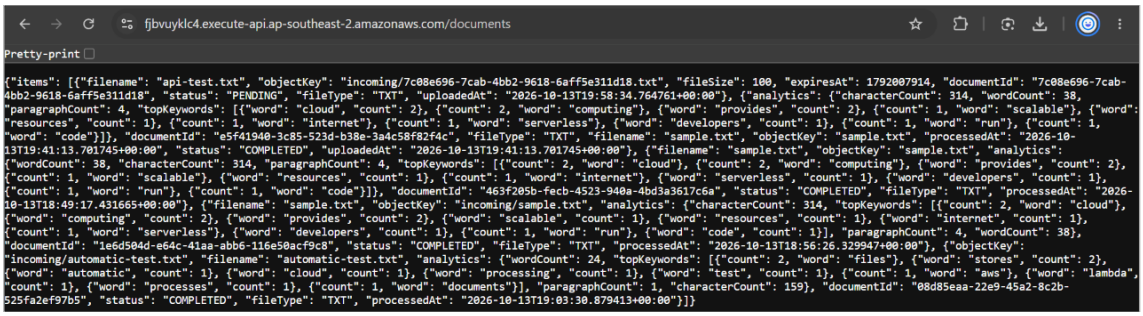


Figure 13: GET /documents response from the deployed API (01:40 IST), showing stored analytics

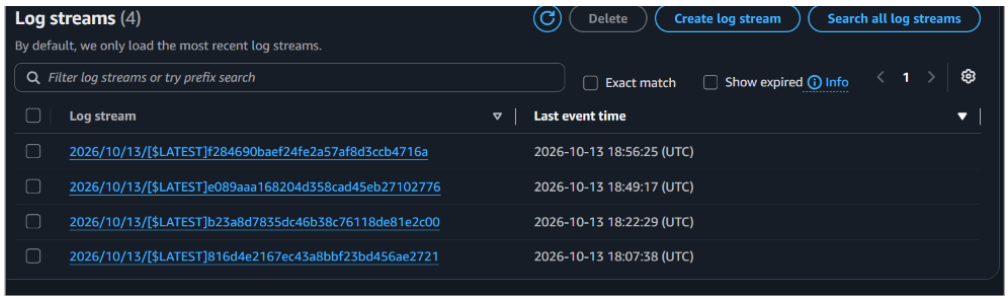


Figure 14: CloudWatch log streams for /aws/lambda/DocumentProcessor (times in UTC)

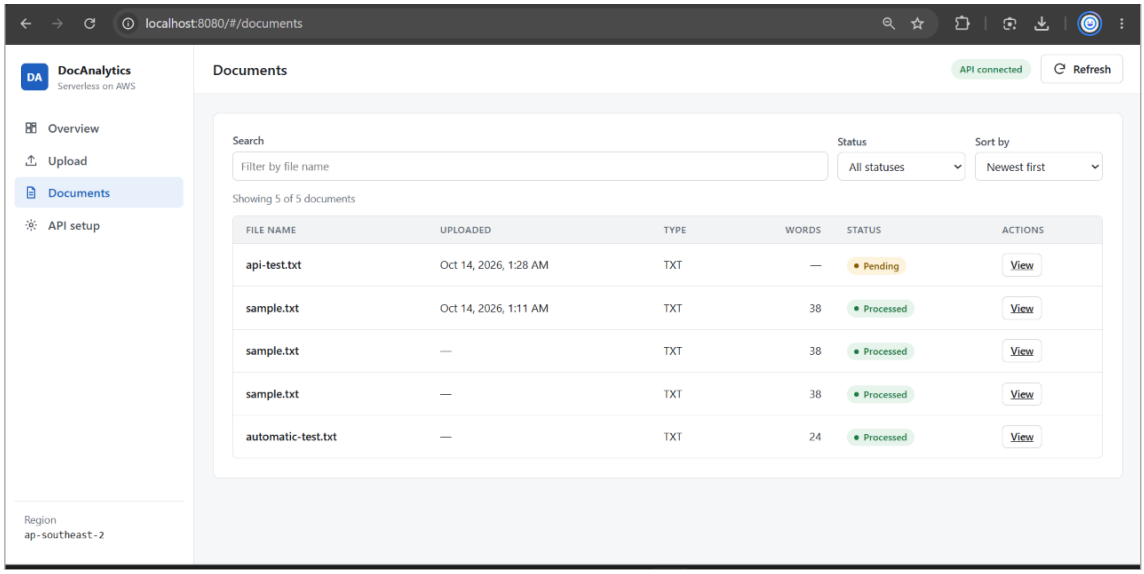


Figure 15: Dashboard Documents view at 02:03 IST: 5 records, 4 processed and 1 pending

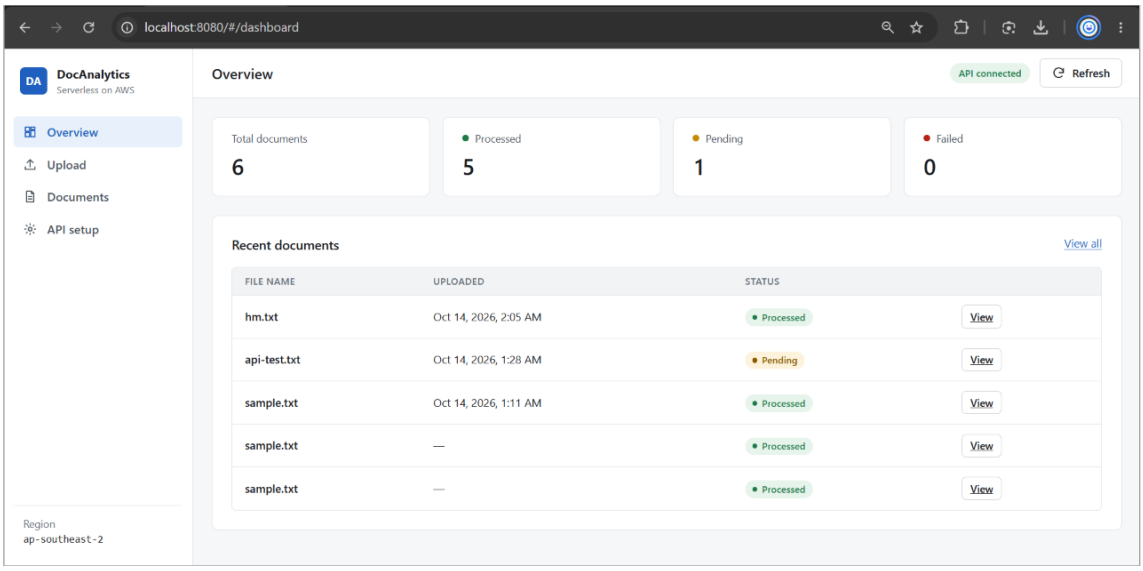


Figure 16: Dashboard Overview at 02:08 IST after the hm.txt upload: 6 total, 5 processed, 1 pending, 0 failed

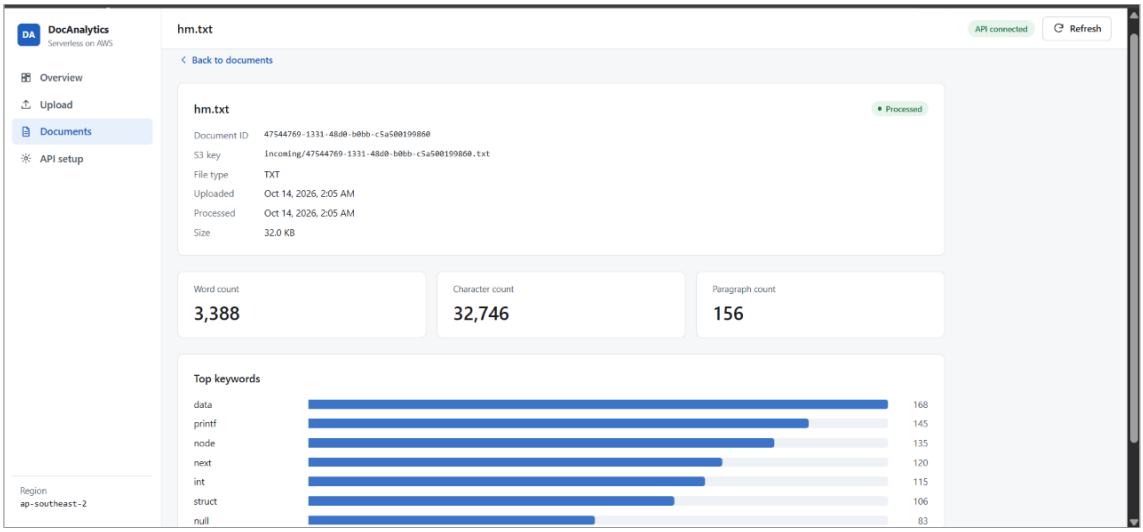


Figure 17: Details view for hm.txt: document ID, S3 key, counts and top keywords

8. Results and discussion

8.1 Demonstrated functionality

The tests show that each part of the platform works on AWS and that the parts work together:

- The S3 event notification invokes DocumentProcessor for TXT files under incoming/ (tests 2, 3 and 8).
- DocumentProcessor reads the object, computes the four analytics and writes them to DynamoDB (tests 1–3, 8).
- DocumentAnalyticsV3 keeps a persistent record for every document, which survives across sessions and is visible through the API (tests 4, 7).
- The HTTP API serves all three routes to the browser with CORS (tests 4–9).
- The full upload workflow works end to end: POST /uploads, presigned PUT, S3 event, processing, DynamoDB update and display in the dashboard (test 8).

8.2 Analytics generated

Table 9 lists the records in DocumentAnalyticsV3 at the end of testing, combining the API output (Figure 13) and the dashboard (Figures 15–17).

Table 9: Records observed in DocumentAnalyticsV3

File	Object key	Created by	Words	Chars	Paras	Status
sample.txt	`sample.txt`	Lambda console test	38	314	4	COMPLETED
sample.txt	`incoming/sample.txt`	S3 console upload	38	314	4	COMPLETED
automatic-test.txt	`incoming/automatic-test.txt`	S3 console upload	24	159	1	COMPLETED
sample.txt	`sample.txt`	Lambda console test	38	314	4	COMPLETED
api-test.txt	`incoming/7c08e696-….txt`	POST /uploads only	–	–	–	PENDING
hm.txt	`incoming/47544769-….txt`	Upload workflow	3,388	32,746	156	COMPLETED

For sample.txt the top keywords were cloud (2), computing (2), provides (2), and scalable, resources, internet, serverless, developers, run and code (1 each). The three sample.txt runs produced identical counts, which shows that the analysis is deterministic. For hm.txt the most frequent terms were data (168), printf (145), node (135), next (120) and int (115), which reflects the C-programming content of that file.

8.3 The pending test record

api-test.txt is still PENDING, and this is the expected result rather than a fault. The record was created by calling POST /uploads directly to test the API. That call writes the PENDING item and returns a presigned URL, but creating a presigned URL does not upload anything: the URL is only a signed permission. No file was sent to the URL, so no object was created under incoming/, S3 emitted no event and DocumentProcessor never ran. The record therefore has no analytics. Its expiresAt attribute marks it for clean-up: if DynamoDB TTL is enabled on that attribute, abandoned PENDING items are deleted automatically [20].

8.4 Issues found during testing

- **Missing CORS origin.** The first retrieval of the API's CORS configuration listed the allowed methods but no allowed origin, so the browser could not read API responses. Adding http://localhost:8080 fixed this, and the dashboard then showed "API connected".
- **Attribute names differ between versions.** The deployed functions store filename, objectKey and a nested analytics map, while the reference code uses fileName, s3Key and flat fields. The dashboard normalises both forms, but the two versions should be brought together.
- **Duplicate records from repeated tests.** Early runs created a new random ID for every invocation, so the same sample.txt appears three times. Deriving the ID from the object key (Section 6.3) prevents this for API uploads.
- **Several table versions.** DocumentAnalytics and DocumentAnalyticsV2 remain from earlier iterations and an earlier environment variable pointed at V2. Only V3 is used now; the old tables can be deleted.
- **Broad execution role.** PowerUserAccess was observed on the development role; the least-privilege policies in backend/iam/ should replace it.

9. Limitations and future scope

9.1 Current limitations

Table 10 separates what is implemented today from improvements that are planned but not built.

Table 10: Implemented features and planned improvements

Area	Implemented now	Planned improvement (not implemented)
Authentication	None: anyone with the API URL can call it	Amazon Cognito user pool with a JWT authorizer; API throttling
IAM	Execution role with PowerUserAccess observed; scoped policies drafted	Attach the least-privilege policies in backend/iam/
File types	TXT (text/plain) only	PDF text extraction before analysis
File size	1 MB limit enforced in the browser; also coded in the API and processor reference code	Larger files with streamed reading
Listing	GET /documents scans the whole table	Query on an index with paginated responses
Error reporting	FAILED status and errorMessage (designed, not exercised)	Show reasons and a retry action in the UI
Operations	CloudWatch logs only	CloudWatch alarms, an on-failure destination for async events, dashboards
Hosting	Frontend served on localhost:8080	Hosted static site with its origin added to CORS

9.2 Future scope

- Support PDF and other formats by extracting text before the existing analysis runs.
- Add user accounts so that each user sees only their own documents.
- Report processing errors clearly in the dashboard, with the reason and an option to retry.
- Raise the 1 MB limit for larger documents while keeping memory use bounded.
- Extend the analytics with sentence counts, average word length and estimated reading time.
- Add production-grade monitoring: alarms on function errors and throttles, and failed-event capture.
- Describe the whole stack as infrastructure as code so it can be recreated in one step.

10. Conclusion

10.1 Conclusion

The project delivered a working serverless document processing and analytics platform on AWS. A private S3 bucket receives TXT uploads through presigned URLs; an S3 ObjectCreated trigger invokes the DocumentProcessor Lambda function, which computes the word, character and paragraph counts and the top 10 keywords; the results are stored with the processing status in the DocumentAnalyticsV3 table; and the DocumentApi function behind the DocumentAnalyticsAPI HTTP API serves them to an HTML, CSS and JavaScript dashboard. Tests on 13–14 October 2026 verified direct processing, S3-triggered processing, the API routes and a complete upload, and showed that a record stays PENDING when only an upload URL is requested. The main remaining gaps are authentication, least-privilege roles in the deployed account, and larger or non-TXT files.

10.2 Key learnings

Table 11: Key learnings

Learning	What it meant in this project
Serverless architecture	Compute exists only per request or event; no server had to be patched or sized
Event-driven design	Processing is decoupled from the browser: S3 events start it, and status fields report progress
Cloud storage	Prefix and suffix filters on S3 events control exactly which objects trigger work
API integration	Presigned URLs keep large uploads out of the API, and CORS must be set on both the API and S3
IAM	A presigned URL is only as powerful as its signer; roles should grant only the actions each function needs
NoSQL databases	Designing from access patterns gives a single-item model keyed by documentId

References

[1] AWS, "What is AWS Lambda?" AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/welcome.html

[2] AWS, "Tutorial: Using an Amazon S3 trigger to invoke a Lambda function," AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/with-s3-example.html

[3] AWS, "Amazon S3 Event Notifications," Amazon S3 User Guide. https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html

[4] AWS, "Uploading objects with presigned URLs," Amazon S3 User Guide. https://docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html

[5] AWS, "Blocking public access to your Amazon S3 storage," Amazon S3 User Guide. https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html

[6] AWS, "API Gateway HTTP APIs," Amazon API Gateway Developer Guide. https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api.html

[7] AWS, "Configure CORS for HTTP APIs in API Gateway," Amazon API Gateway Developer Guide. https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-cors.html

[8] AWS, "What is Amazon DynamoDB?" Amazon DynamoDB Developer Guide. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

[9] AWS, "NoSQL design for DynamoDB," Amazon DynamoDB Developer Guide. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-general-nosql-design.html

[10] AWS, "Defining Lambda function permissions with an execution role," AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html

[11] AWS, "Security best practices in IAM," AWS IAM User Guide. https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

[12] AWS, "Sending Lambda function logs to CloudWatch Logs," AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/monitoring-cloudwatchlogs.html

[13] AWS, "AWS Serverless Multi-Tier Architectures with Amazon API Gateway and AWS Lambda," AWS Whitepaper.

https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/welcome.html

[14] AWS, "Define Lambda function handler in Python," AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/python-handler.html

[15] AWS, "Invoking a Lambda function asynchronously," AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/invocation-async.html

[16] Python Software Foundation, "re — Regular expression operations," Python 3 documentation. https://docs.python.org/3/library/re.html

[17] Mozilla, "Cross-Origin Resource Sharing (CORS)," MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

[18] AWS, "Using cross-origin resource sharing (CORS)," Amazon S3 User Guide. https://docs.aws.amazon.com/AmazonS3/latest/userguide/cors.html

[19] Python Software Foundation, "collections — Container datatypes," Python 3 documentation. https://docs.python.org/3/library/collections.html

[20] AWS, "Using time to live (TTL) in DynamoDB," Amazon DynamoDB Developer Guide. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html

[21] AWS, "Boto3 documentation." https://boto3.amazonaws.com/v1/documentation/api/latest/index.html

Appendix A: Evidence sources

All screenshots are unedited captures from the project's AWS account and local dashboard, except that three are cropped and two have the AWS account ID covered. Capture times are local time (IST).

Table 12: Evidence index

Figure	Shows	Captured
2	S3 bucket created	13 Oct 2026, 23:16
3	DocumentProcessor overview (ARN redacted)	13 Oct 2026, 23:34
4	Deployed DocumentProcessor handler	14 Oct 2026, 01:10
5	S3 event notification	14 Oct 2026, 00:31
6	DynamoDB tables	14 Oct 2026, 00:18
7	IAM execution role (account ID redacted)	14 Oct 2026, 00:12
8	HTTP API created	14 Oct 2026, 01:33
9	API CORS configuration	14 Oct 2026, 01:35
10	Direct Lambda test succeeded (cropped)	14 Oct 2026, 00:19
11	Console upload of automatic-test.txt	14 Oct 2026, 00:33
12	CORS configuration via CloudShell (cropped)	14 Oct 2026, 01:59
13	GET /documents JSON response (cropped)	14 Oct 2026, 01:40
14	CloudWatch log streams	14 Oct 2026, 00:27
15	Dashboard Documents view	14 Oct 2026, 02:03
16	Dashboard Overview	14 Oct 2026, 02:08
17	Details view for hm.txt	14 Oct 2026, 02:09